

LiDAR 01: Introduction



Networked Systems Lab @ Chung-Ang University

Kyungmin Kim
Chung-Ang University
kyungddin@cau.ac.kr
Jul. 17th 2025

Overview



- In this lecture, you will learn..
 1. What is LiDAR?
 2. How to connect Ouster LiDAR?
 3. Various LiDAR Frameworks Introduction
 4. LiDAR Visualization
 5. Open3D: Nice and Easy Framework

Notice!



- Recommend to work in Linux environment
 - Many LiDAR Frameworks work well in Linux environment.
- For version management
 - Anaconda is recommended.
 - You will experience many package problems..
- Some steps may change in the future
 - You need to check official document.



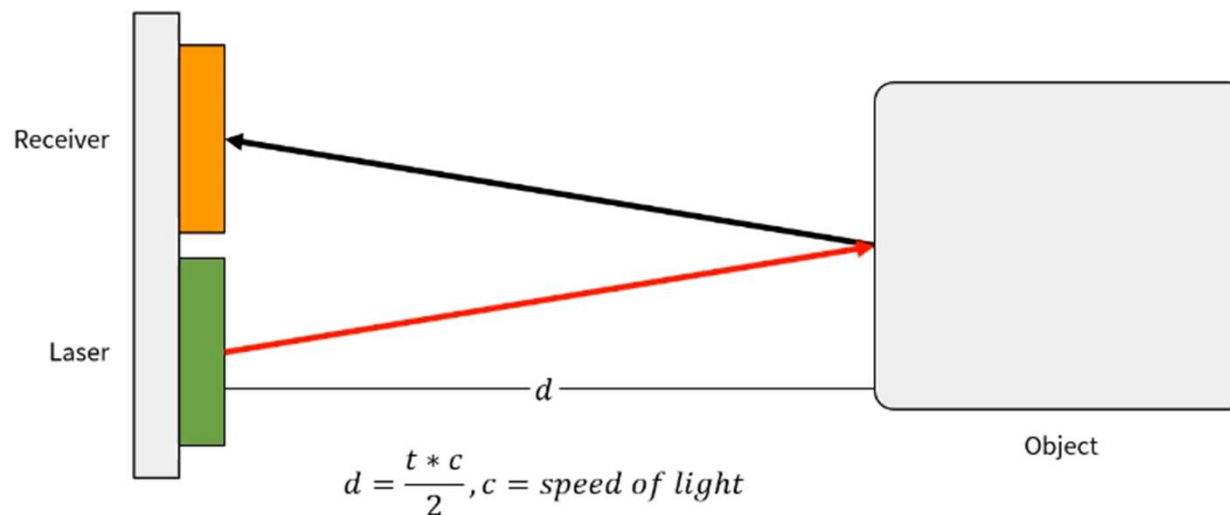


What is LiDAR?

LiDAR Sensor Introduction

LiDAR Introduction

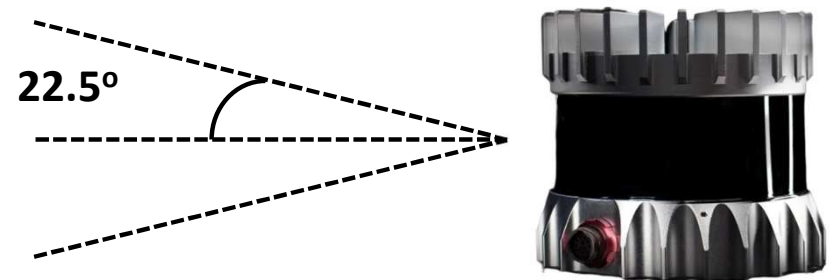
- **Li**ght **D**etection **A**nd **R**anging
 - a.k.a **LiDAR**
 - Sensor that uses **laser reflection**.
 - Through this, It can measure distance & shape.
- LiDAR collects various **physical quantity**
 - Distance, Reflectivity, Angle, Return Number...



LiDAR Introduction: Ouster OS1-32



- **Ouster** LiDAR Sensor
 - Powerful and Expensive LiDAR Sensor.
 - [Lecture Standard: Ouster OS1-32](#)
- **Ouster OS1-32 Spec**
 - Maximum Distance: 120M
 - Vertical Angle: $\pm 22.5^\circ$
 - Horizontal Angle: 360°



120m



Now let's connect Ouster LiDAR!

LiDAR Connection Step

LiDAR Connection Step



- To use Ouster LiDAR
 - Step1: Hardware Connection
 - Step2: Network Setting





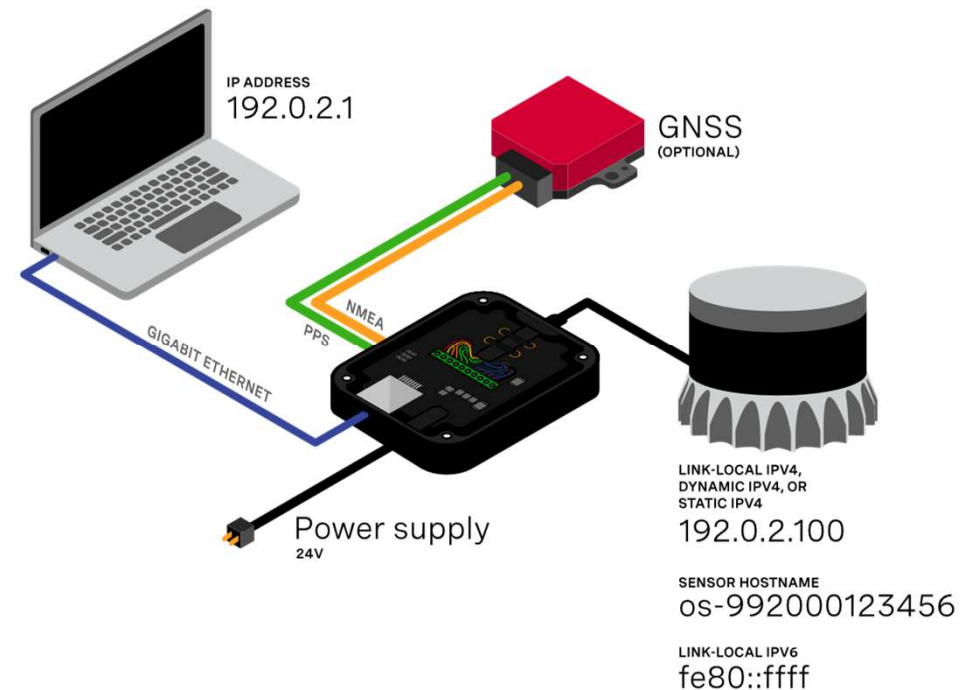
Step 1

Hardware Connection

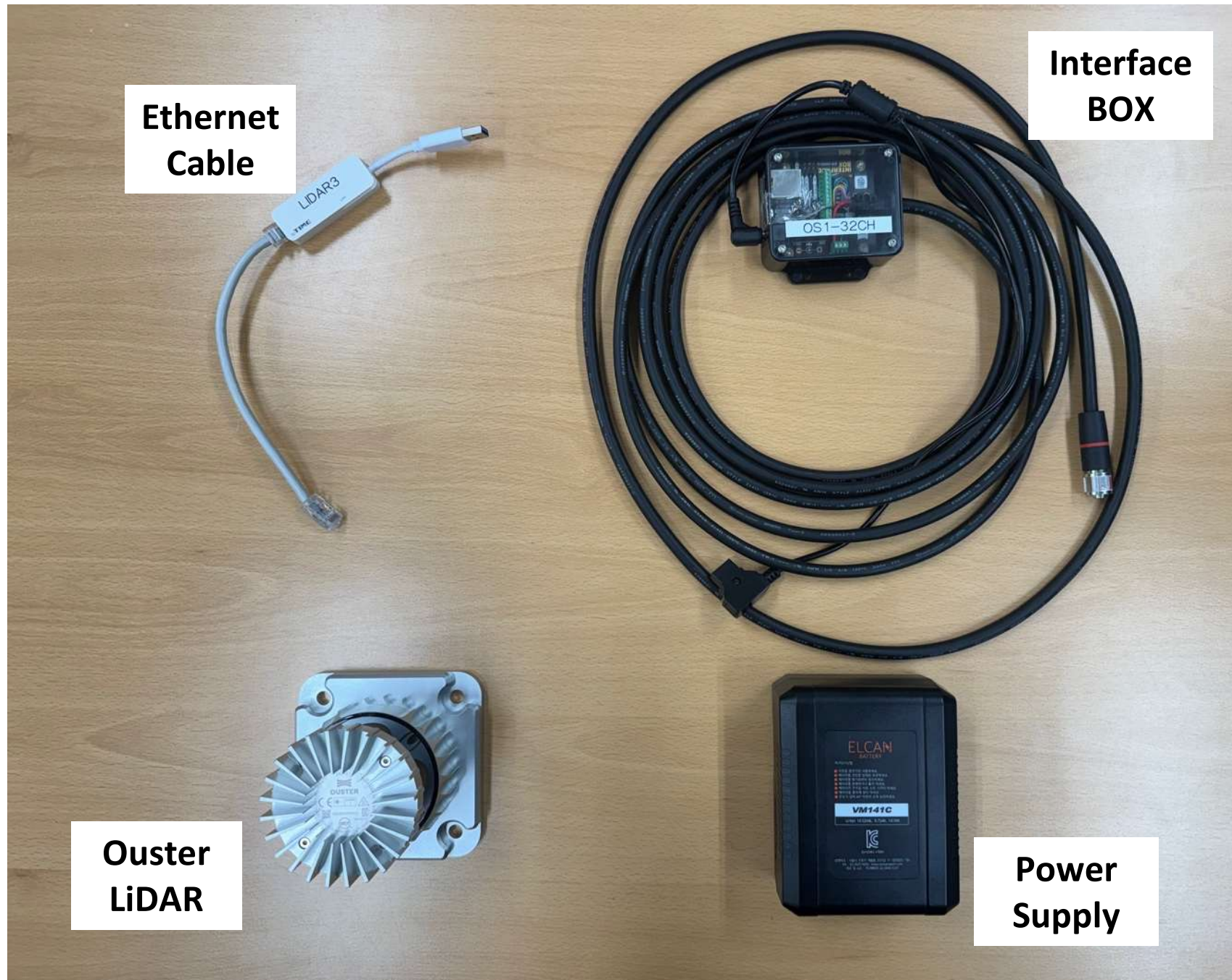
Hardware Connection



- Hardware Essential Component
 - PC
 - Interface Box
 - Transmit data between PC and Sensor
 - Ethernet Cable
 - Cable between PC and Interface
 - Ouster LiDAR
 - Power Supply/Battery



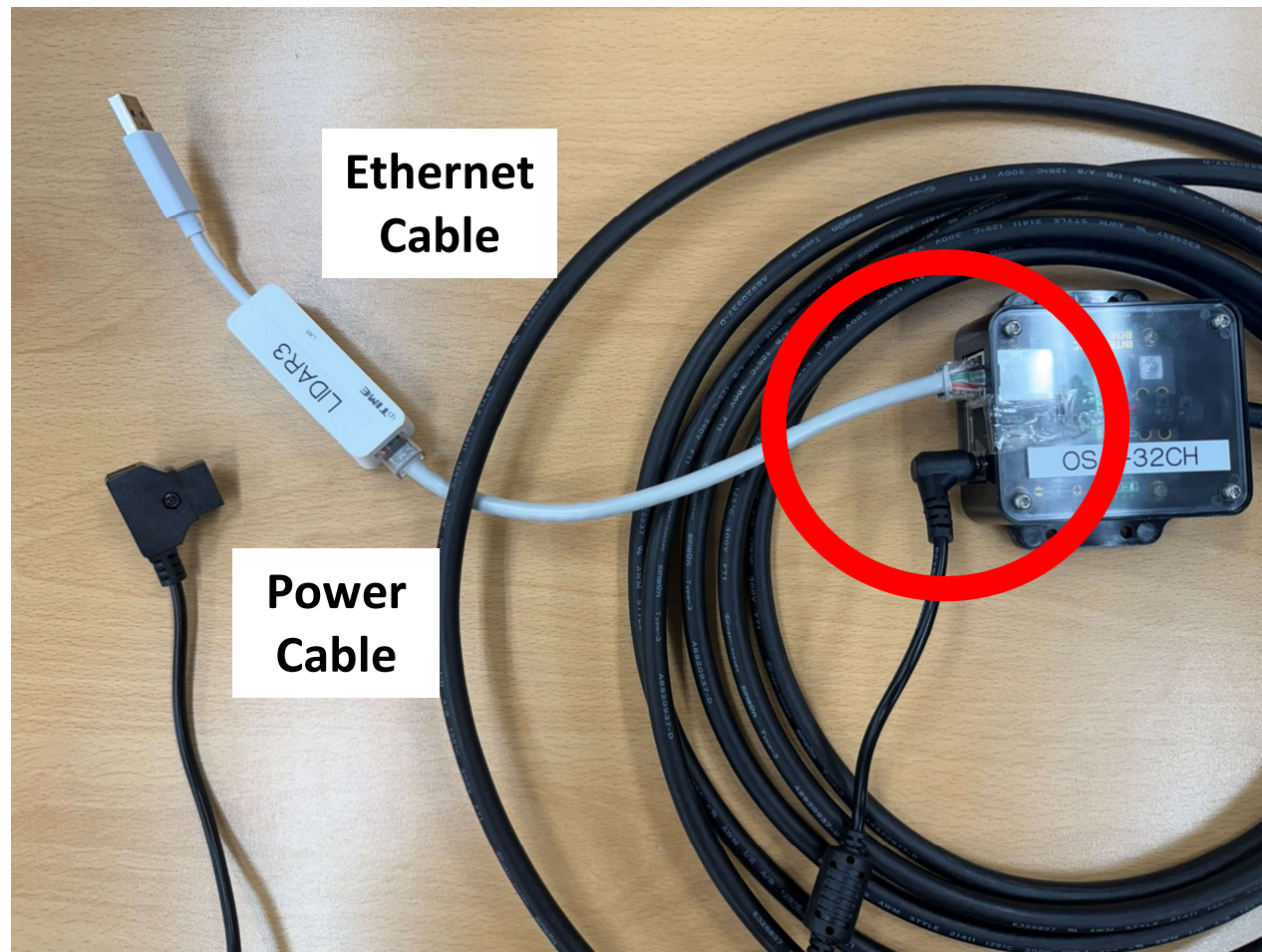
Hardware Connection



Hardware Connection



- LiDAR Connection Step 1
 - Connect **Ethernet cable** & **Power cable** to **Interface Box**.



Hardware Connection



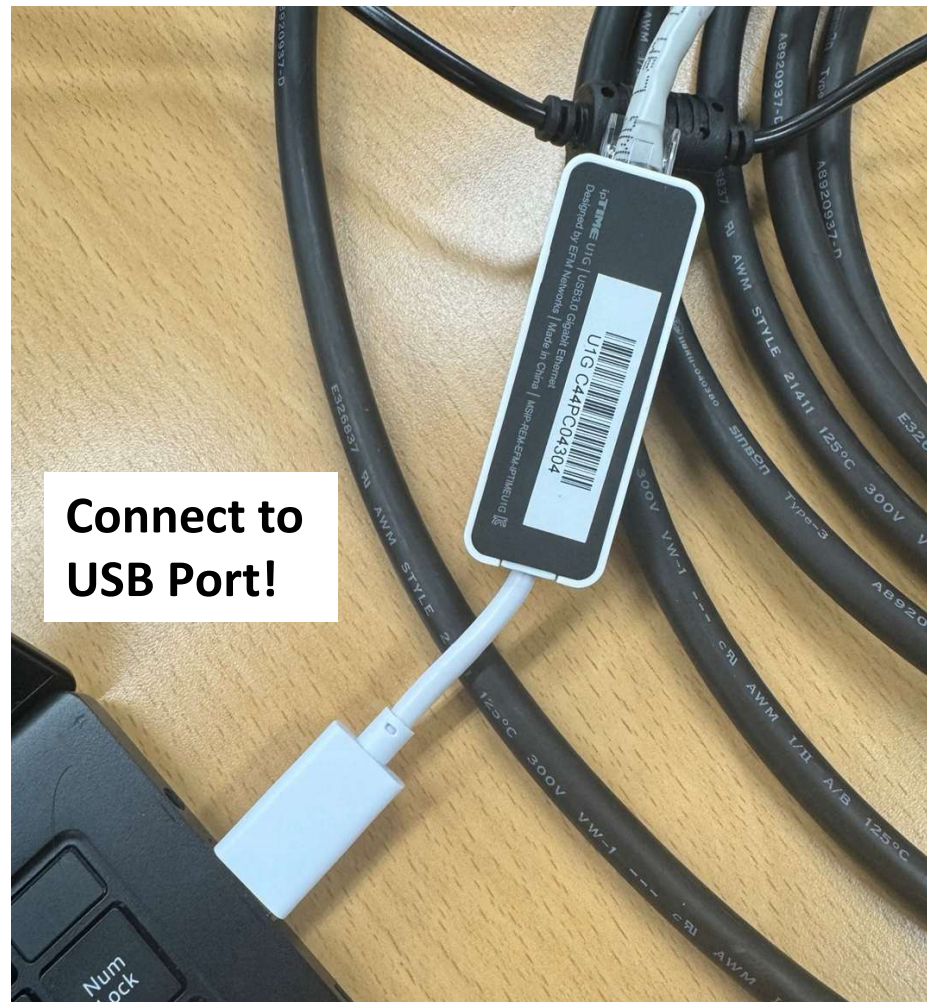
- LiDAR Connection Step 2
 - Connect **Power cable** to **Power Supply(Battery)**.
 - A **green light** turns on when the power is on



Hardware Connection



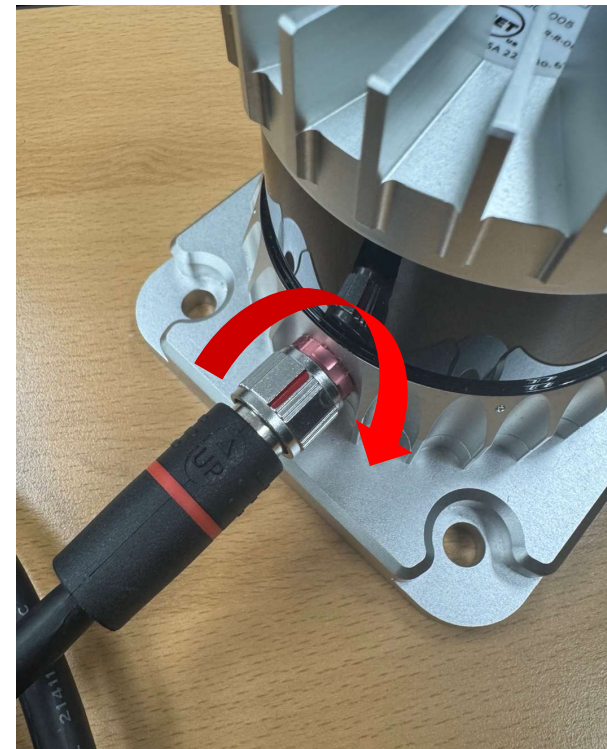
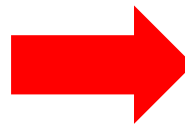
- LiDAR Connection Step 3
 - Connect **Ethernet** cable to **PC**.



Hardware Connection



- LiDAR Connection Step 4
 - Connect **Interface Box** to **Ouster LiDAR**.
 - Align **Red line** to “UP” text.
 - Turn clockwise until you hear a “Click!” sound.

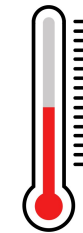


Hardware Connection

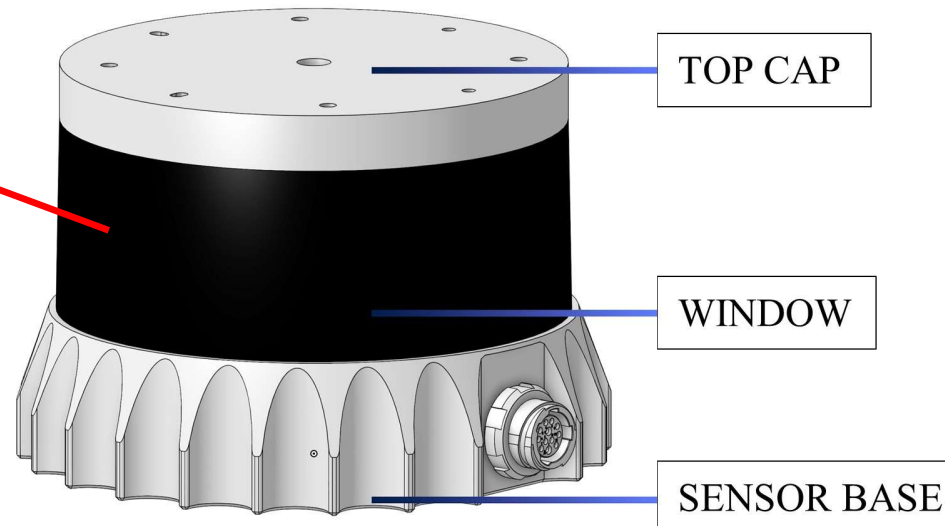


■ OS1-32 Cautions

- Don't touch the **Window!**
 - There's Laser out and Receiver.
- Watch out for LiDAR **Overheating!**
 - Recommended usage time is **4 hours**.



Need to **cool down**
After 4 hours! ☹️





Step 2

Ubuntu Network Setting

Ubuntu Network Setting: PC Network

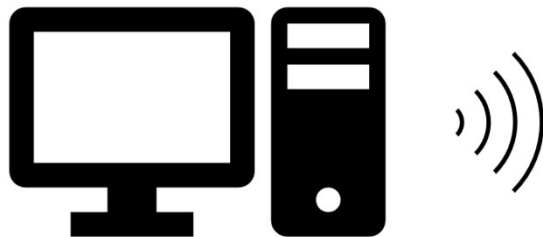


LiDAR Communication

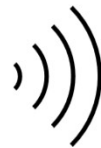
- LiDAR uses **UDP Protocol**
 - Set Ethernet **static IP address**
 - Assign **DHCP IP address** to LiDAR sensor

Why use the DHCP protocol?

1. To simplify configuration for multiple LiDAR connections
2. To prevent IP address conflicts



PC
Static IP



UDP Network
192.168.0.XX



Sensor
DHCP IP

Ubuntu Network Setting: PC Network

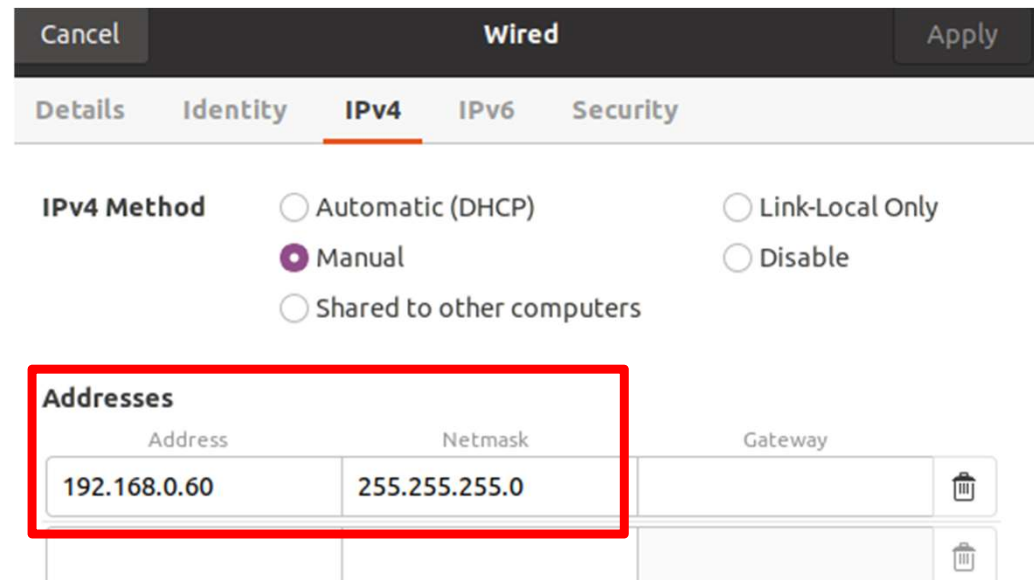
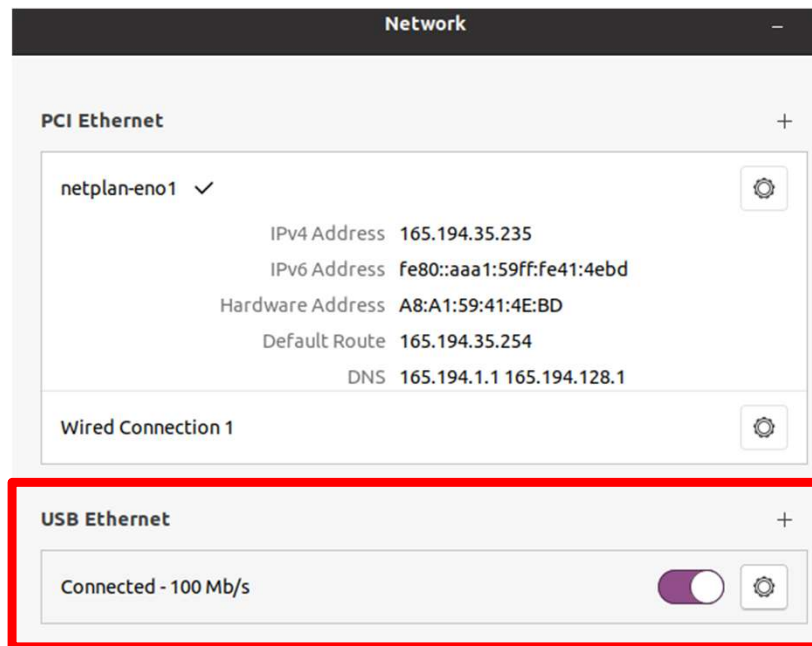
■ PC Network Setting

1. Linux Setting: “Network Setting – USB Ethernet – IPv4”

- Select **Static** and enter **IP Address and Netmask**.

2. Address: 192.168.0.XX (XX is your choice; 0 ~ 100)

Netmask: 255.255.255.0



Ubuntu Network Setting: PC Network



- PC Network Setting

3. \$ ifconfig

to check network setting.

- Check network setting and **Device name!** (for Sensor Network)

```
>_
(km) nsl@nsl:~$ ifconfig
eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 165.194.35.235 netmask 255.255.255.0 broadcast 165.194.35.255
    inet6 fe80::aaa1:59ff:fe41:4ebd prefixlen 64 scopeid 0x20<link>
    ether a8:a1:59:41:4e:bd txqueuelen 1000 (Ethernet)
    RX packets 4257251 bytes 953077731 (953.0 MB)
    RX errors 0 dropped 234116 overruns 0 frame 0
    TX packets 160711 bytes 46019290 (46.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 16 memory 0xa2200000-a2220000

enx588694fda289: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.60 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::16d8:c53e:e2b8:d434 prefixlen 64 scopeid 0x20<link>
    ether 58:86:94:fd:a2:89 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Ubuntu Network Setting: Sensor Network



■ Sensor Network Setting

1. `$ sudo dnsmasq -C /dev/null -kd -F 192.168.0.0,192.168.0.100 -i <device_name> -bind-dynamic`

- <device name> is your device name.

2. Check Terminal whether DHCP IP is addressed.

```
dnsmasq: started, version 2.90 cachesize 150
dnsmasq: compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN DHCP DHCPv6 no-Lua TFTP conntac
k ipset no-nftset auth cryptohash DNSSEC loop-detect inotify dumpfile
dnsmasq-dhcp: DHCP, IP range 192.168.0.0 -- 192.168.0.100, lease time 1h
dnsmasq-dhcp: DHCP, sockets bound exclusively to interface enx588694fda289
dnsmasq: reading /etc/resolv.conf
dnsmasq: using nameserver 127.0.0.53#53
dnsmasq: read /etc/hosts - 8 names
dnsmasq-dhcp: DHCPDISCOVER(enx588694fda289) bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPOFFER(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPREQUEST(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPACK(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5 os-662333000005
dnsmasq-dhcp: DHCPREQUEST(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPACK(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5 os-662333000005
dnsmasq: error binding DHCP socket to device enx588694fda289
dnsmasq-dhcp: DHCPDISCOVER(enx588694fda289) bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPOFFER(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPDISCOVER(enx588694fda289) bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPOFFER(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPREQUEST(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5
dnsmasq-dhcp: DHCPACK(enx588694fda289) 192.168.0.49 bc:0f:a7:00:6a:b5 os-662333000005
```

Ubuntu Network Setting: Shell Script



■ Shell Script Guide

1. It's annoying to do this every time. We use **Shell Script!**
2. Make **dhcp.sh** file to your home directory.
3. Open your text editor(e.g. vim, nano) and write

```
#!/bin/bash
```

```
sudo dnsmasq -C/dev/null -kd -F 192.168.0.0,192.168.0.100 -i [device_name] -bind-dynamic
```

```
터미널

#!/bin/bash

# 1. dnsmasq 실행해서 LiDAR에 IP 자동 할당
sudo dnsmasq -C/dev/null -kd -F 192.168.0.0,192.168.0.100 -i enx588694fda289 --bind-dynamic
~
```


Ubuntu Network Setting: Shell Script



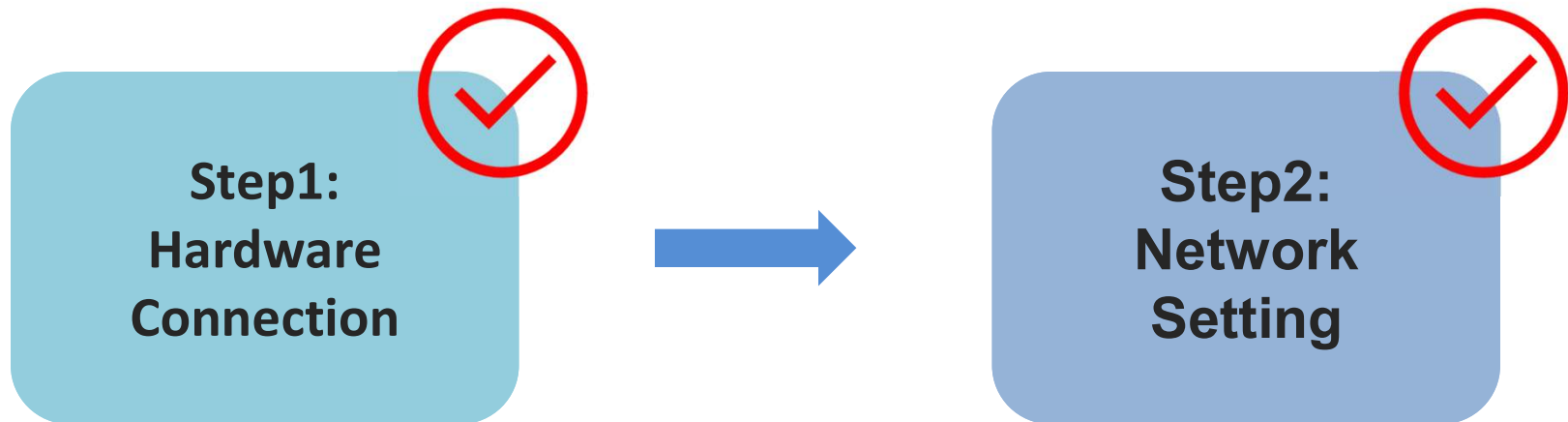
- Shell Script Guide

4. `$ sudo chmod _x ~/dhcp.sh`

to add execution permission.

5. `$ ./dhcp.sh`

to get DHCP IP quickly!



Complete!!

Ubuntu Network Setting: Sensor Network



- DHCP Re-Address

- When you close the terminal, power off the sensor or whatever.

there's still DHCP address in the process..

- You must kill the process by

```
$ sudo pkill -9 dnsmasq
```

Process is still alive!

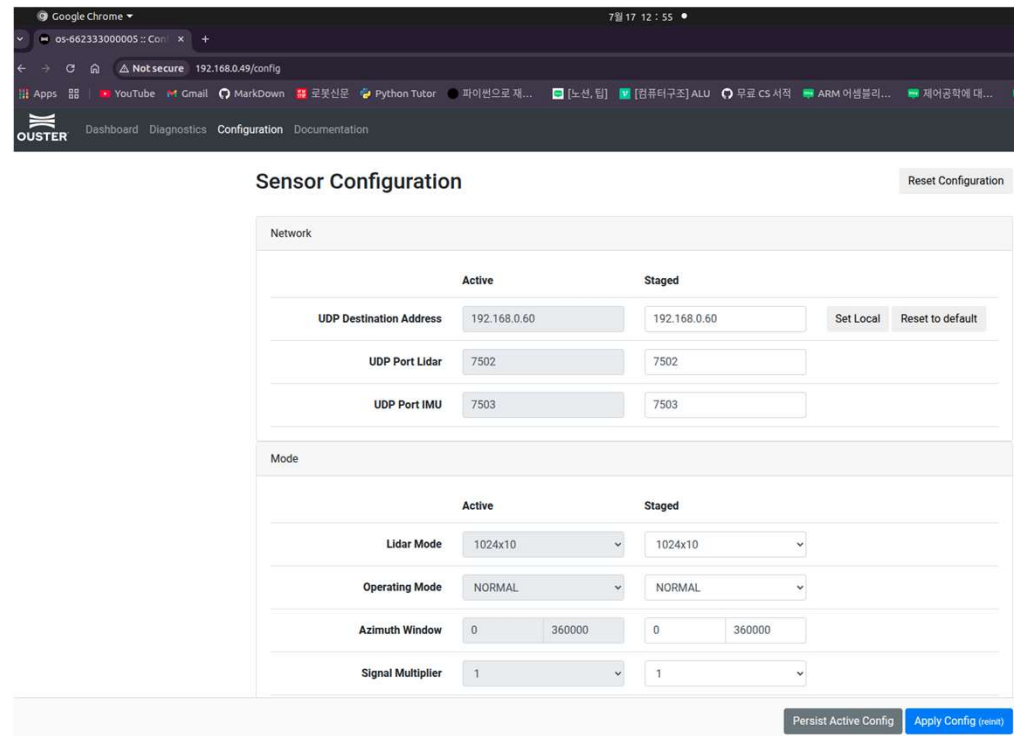
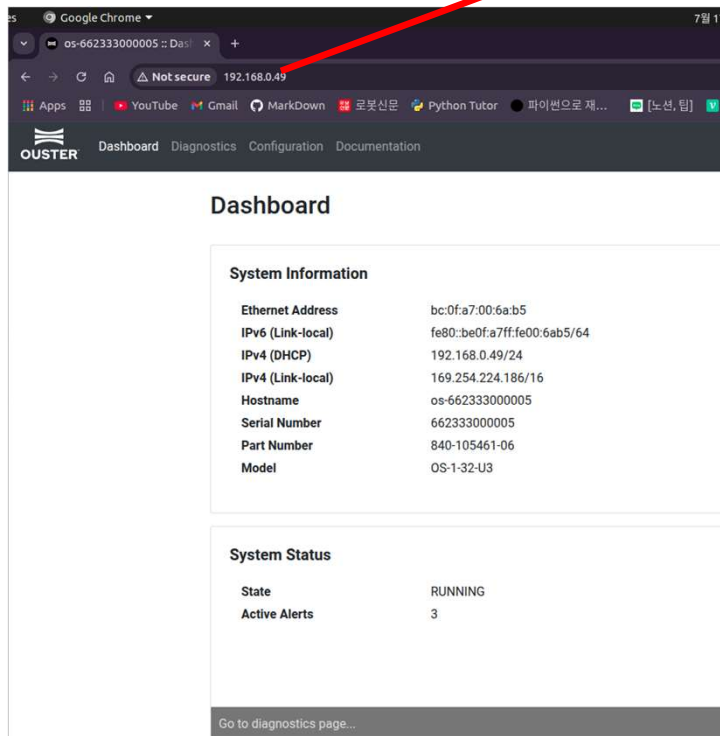
```
dnsmasq: failed to create listening socket for 127.0.0.1: Address already in use
(km) nsl@nsl:~$ sudo pkill -9 dnsmasq
(km) nsl@nsl:~$ ./dhcp.sh
dnsmasq: started, version 2.90 cachesize 150
dnsmasq: compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN DHCP DHCPv6 no-Lua TFTP conntac
k ipset no-nftset auth cryptohash DNSSEC loop-detect inotify dumpfile
dnsmasq-dhcp: DHCP, IP range 192.168.0.0 -- 192.168.0.100, lease time 1h
dnsmasq-dhcp: DHCP, sockets bound exclusively to interface enx588694fda289
dnsmasq: reading /etc/resolv.conf
dnsmasq: using nameserver 127.0.0.53#53
dnsmasq: read /etc/hosts - 8 names
```

Ubuntu Network Setting: Browser Network Setting



■ Internet Browser Network Setting

1. After connection, you can use **browser network setting**.
2. Open your internet browser and enter your sensor address.
3. You can check **sensor information** and **configure sensor setting**!





To help you control LiDAR

LiDAR Frameworks

LiDAR Frameworks



■ LiDAR Frameworks

- There are many **LiDAR Frameworks** to help you control the LiDAR.
 - Hardware Company's SDK(Software Development Kit)
 - ROS
 - Programming Library
- To send sensor data to your program, **Ouster SDK** is a good choice.

		
SDK	ROS	Library

LiDAR Frameworks



■ LiDAR Frameworks Example

- Ouster SDK
 - Official SDK for Ouster LiDAR
- ROS
 - Middleware for Robotics and LiDAR Sensor
- Open3D
 - Python/C++ Point Cloud Library
- PCL
 - C++ Point Cloud Library





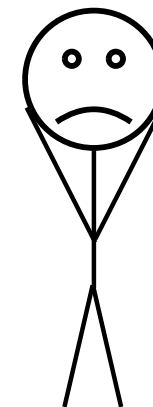
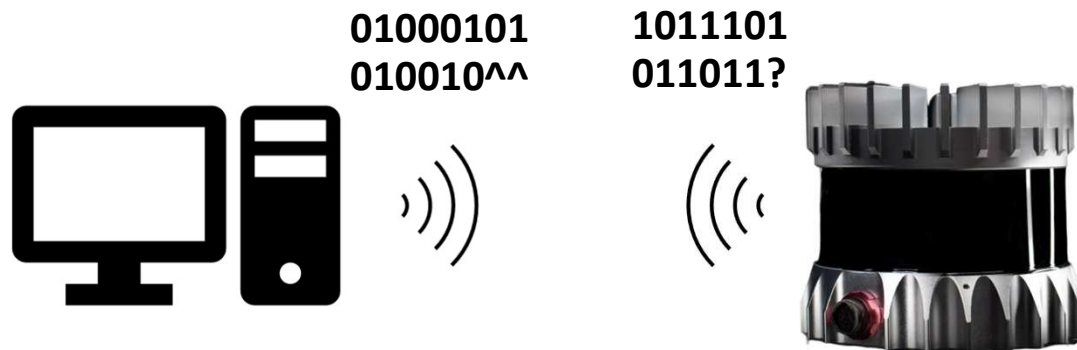
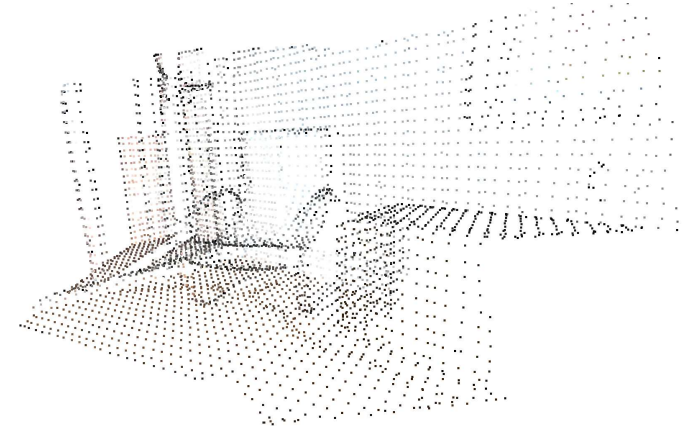
Visualize Sensor's Data

Point Cloud Visualization

Point Cloud Visualization



- Point Cloud
 - Converted LiDAR data
 - Each Point has (X,Y,Z) spatial information.
- Point Cloud Visualization
 - Connection is complete. But we don't know what exactly happened inside.
 - We need to visualize Point cloud to see what is going on!



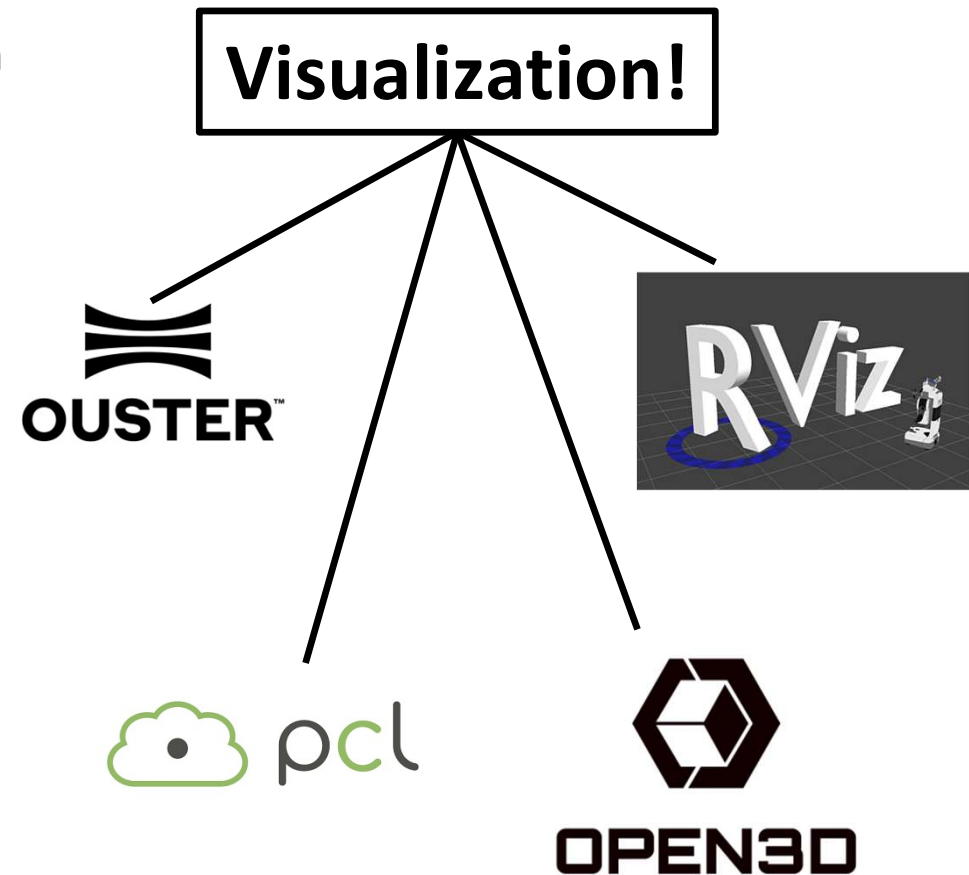
OMG!!

Point Cloud Visualization



■ Point Cloud Visualization Tool Examples

- Ouster Studio
 - Official Ouster Visualization Program
 - Easiest way to visualize Point Cloud!
- Rviz
 - ROS Visualization
- PCL
 - C++ Point Cloud Library
- Open3D
 - Python Point Cloud Library

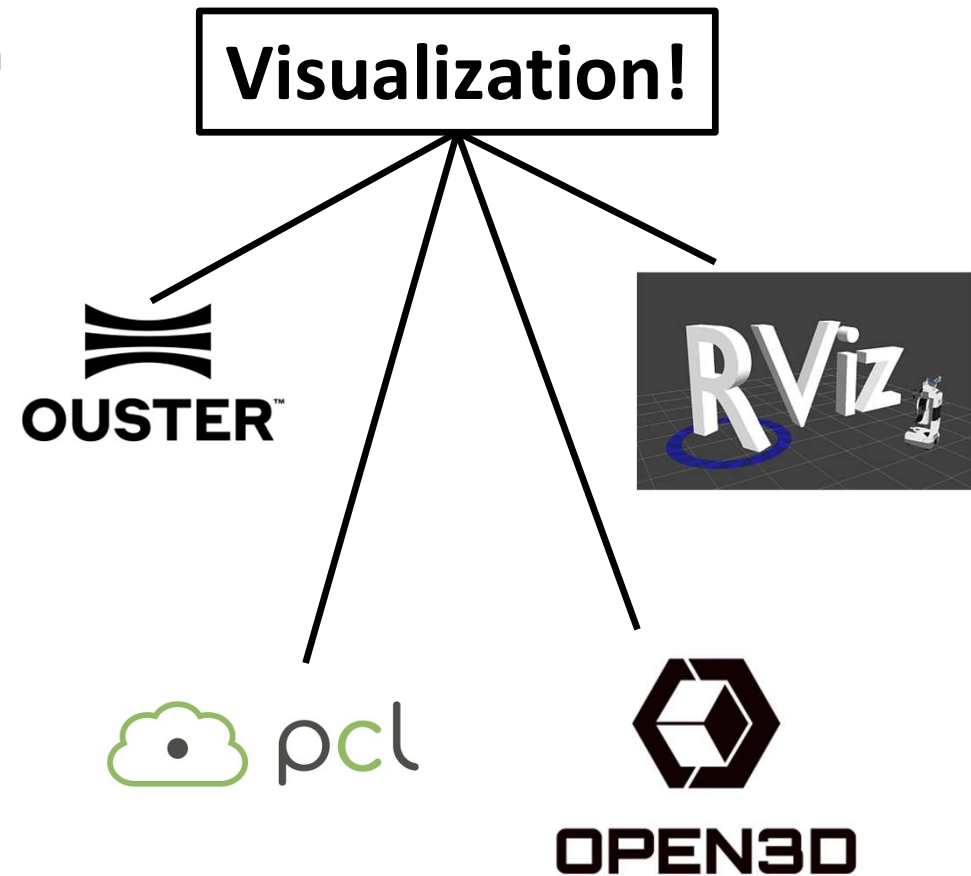


Point Cloud Visualization



■ Point Cloud Visualization Tool Examples

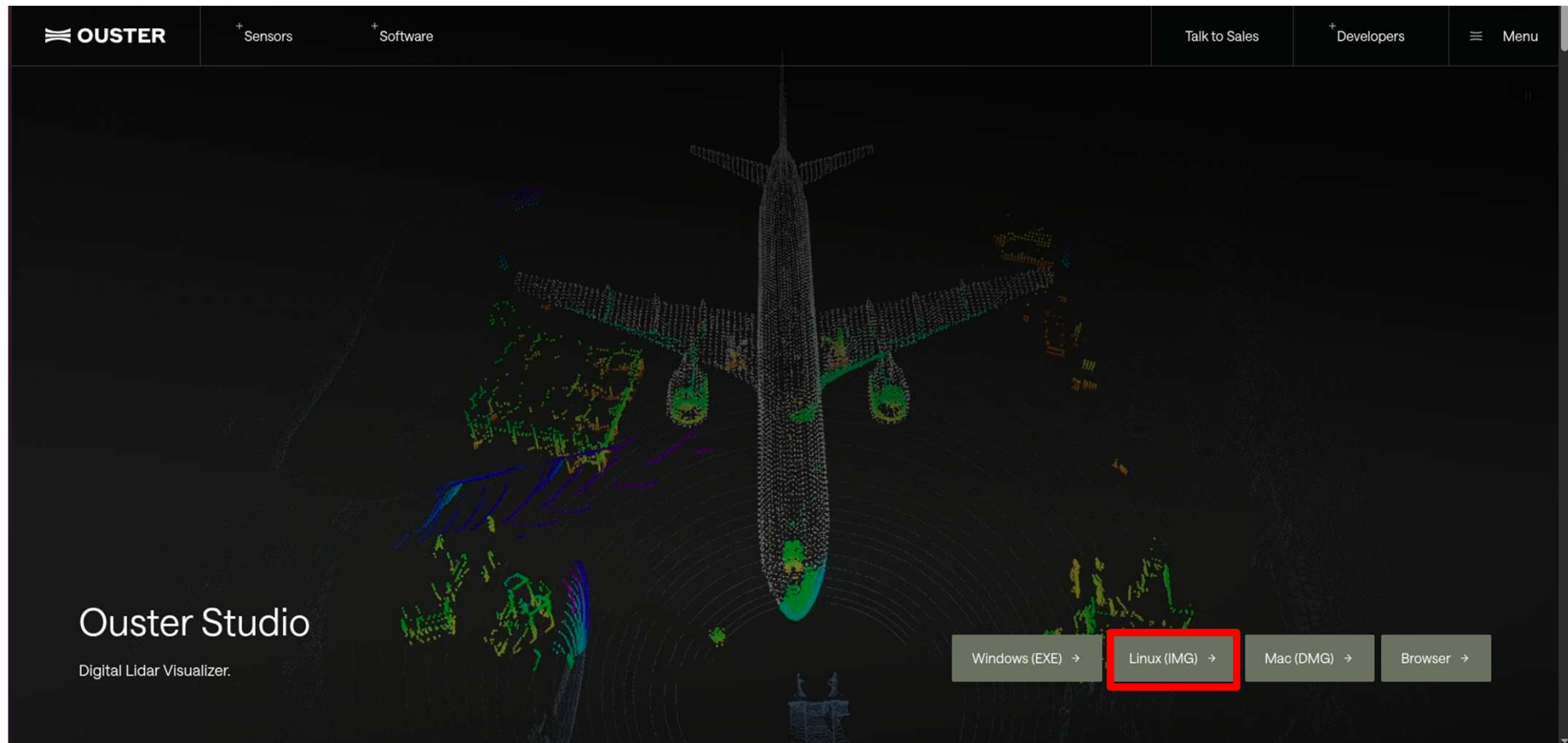
- Ouster Studio
 - Official Ouster Visualization Program
 - Easiest way to visualize Point Cloud!
(We use this program!)
- Rviz
 - ROS Visualization
- PCL
 - C++ Point Cloud Library
- Open3D
 - Python Point Cloud Library



Point Cloud Visualization: Ouster Studio



- How to use Ouster Studio?
 1. Go to Ouster Studio web page and download **Linux(IMG)** file.
 - <https://ouster.com/ko/products/software/ouster-studio>



Point Cloud Visualization: Ouster Studio



- How to use Ouster Studio?
 2. “Properties – Permissions – Check Allow executing file as program”
 3. Run the Image file.

The screenshot shows the 'OusterStudio-2.1.0.AppImage Properties' dialog box with the 'Permissions' tab selected. The dialog has a dark header bar with the title and a close button. Below the header, there are two tabs: 'Basic' and 'Permissions'. The 'Permissions' tab is active, showing settings for Owner, Access, Group, Access, Others, Access, Execute, and Security context. The 'Execute' section is highlighted with a red box, showing a checked checkbox for 'Allow executing file as program'.

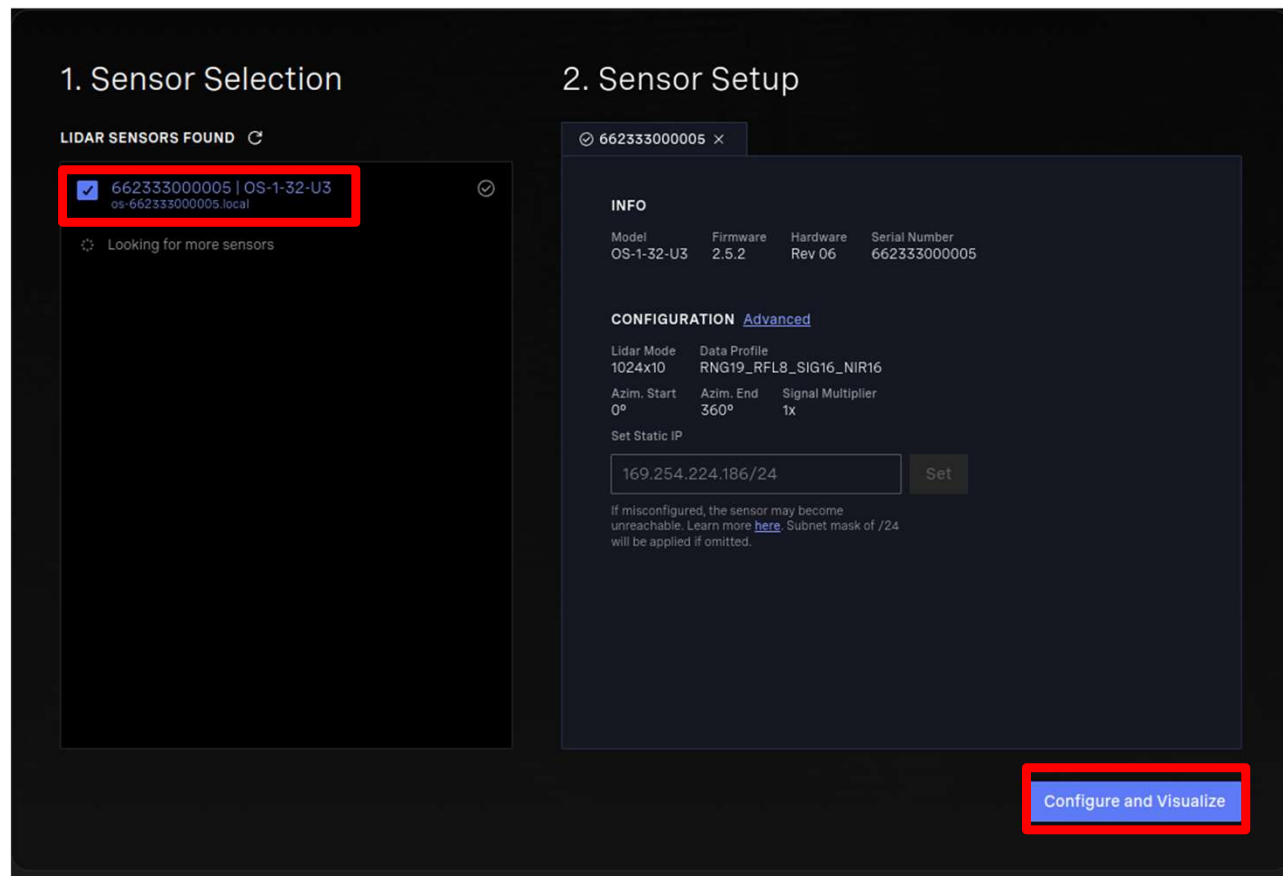
Category	Value
Owner:	Me
Access:	Read and write
Group:	nsl
Access:	Read and write
Others	
Access:	Read-only
Execute:	☒ Allow executing file as program
Security context:	unknown

Point Cloud Visualization: Ouster Studio

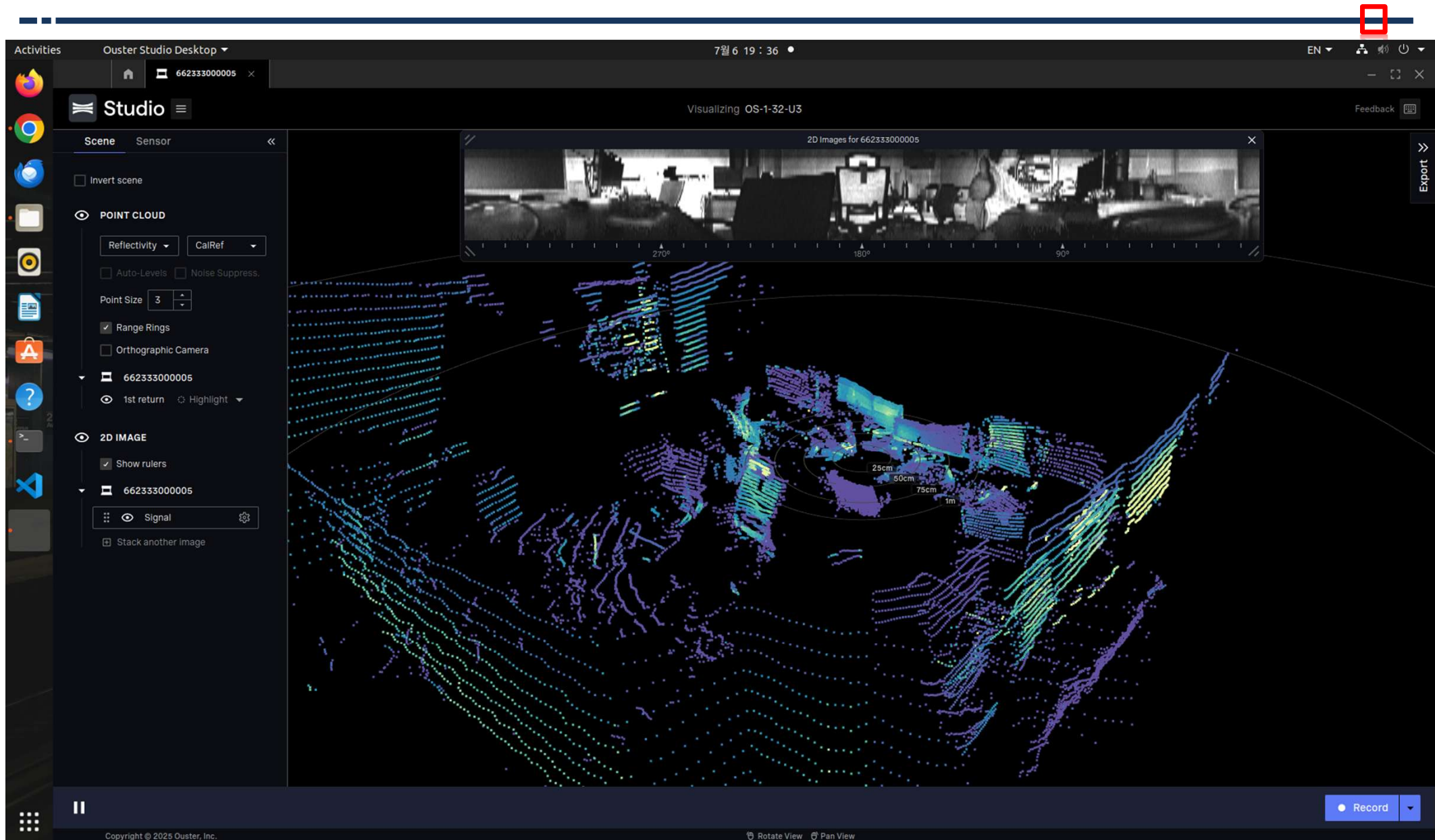


- How to use Ouster Studio?

4. Select your sensor and click “configure and Visualize”.



Point Cloud Visualization: Ouster Studio



Now you can see Point Cloud!



Utilize Point Cloud Data

Ouster SDK & Open3D Introduction

Ouster SDK



- Ouster SDK
 - SDK that **reads, parses and visualizes** the sensor data
 - Usually use to read sensor data
 - Recommend to use other libraries to process the point cloud



OUSTER SDK

Get Sensor Data

Open3D or PCL

Process Point Cloud
Visualization

Open3D Library



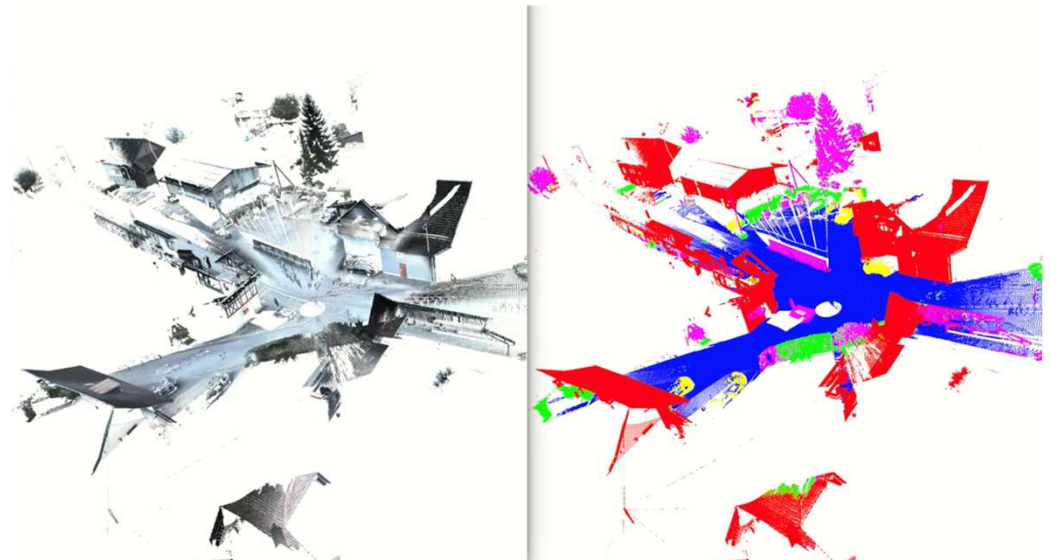
- Python/C++ library to process 3D data like Point Cloud

- Recommend Python language.



- Key Features

- Visualization
- Voxel Downsampling
- Point Cloud Painting
- Point Cloud distance calculation
- Bounding Volume
- Segmentation Plane
- Etc..



Ouster SDK & Open3D

Library Installation

- Open Linux terminal and command

```
$ pip install ouster-sdk
```

```
$ pip install open3d
```

```
(km) nsl@nsl:~$ pip install ouster-sdk \
> pip install open3d
```

To check installation and version

- Command

```
$ pip show ouster-sdk
```

```
$ open3d --version
```

```
(km) nsl@nsl:~$ pip show ouster-sdk
Name: ouster_sdk
Version: 0.15.0
Summary: Ouster Sensor SDK
Home-page: https://github.com/ouster-lidar/ouster-sdk
Author: Ouster Sensor SDK Developers
Author-email: oss@ouster.io
License: BSD 3-Clause License
Location: /home/nsl/anaconda3/envs/km/lib/python3.10/site-packages
Requires: click, importlib-metadata, kiss-icp, laspy, matplotlib, numpy,
pytable, psutil, requests, rosbags, scipy, threadpoolctl
Required-by:
(km) nsl@nsl:~$ open3d --version
*****
* Open3D: A Modern Library for 3D Data Processing *
*
* Version 0.19.0
* Docs https://www.open3d.org/docs
* Code https://github.com/isl-org/Open3D
*****
Open3D 0.19.0
```


Ouster SDK & Open3D: Example



```
scan_one_frame.py > ...
1  # Module Import
2  from ouster.sdk import client
3  from ouster.sdk.client import Scans
4  from ouster.sdk import open_source
5  from ouster.sdk.core import XYZLut
6  import open3d as o3d
7  import numpy as np
8
9  # Ouster LiDAR 불러오기
10 HOST = "192.168.0.49"
11 source = open_source(HOST) # 제너레이터로, 매 프레임마다 스캔 데이터 생성
12
13 # 첫번째 LiDAR의 metadata를 가져오고, 이것의 Range Data 추출
14 metadata = source.metadata[0]
15 xyz_lut = XYZLut(metadata, False)
16
17 for scans in source:
18     # 첫번째 LiDAR의 스캔데이터
19     scan = scans[0]
20     xyz = xyz_lut(scan)
21
22     # Open3D로 시각화
23     pcd_np = xyz.reshape(-1, 3) # Range Data 통해 Point Cloud 생성
24     pcd = o3d.geometry.PointCloud() # 포인트클라우드 객체 생성
25     pcd.points = o3d.utility.Vector3dVector(pcd_np) # open3D 벡터 변환
26     o3d.visualization.draw_geometries([pcd]) # 시각화
27
28     break
29
30
```

1. Module Import

>> ouster.sdk and open3D

2. Ouster.sdk

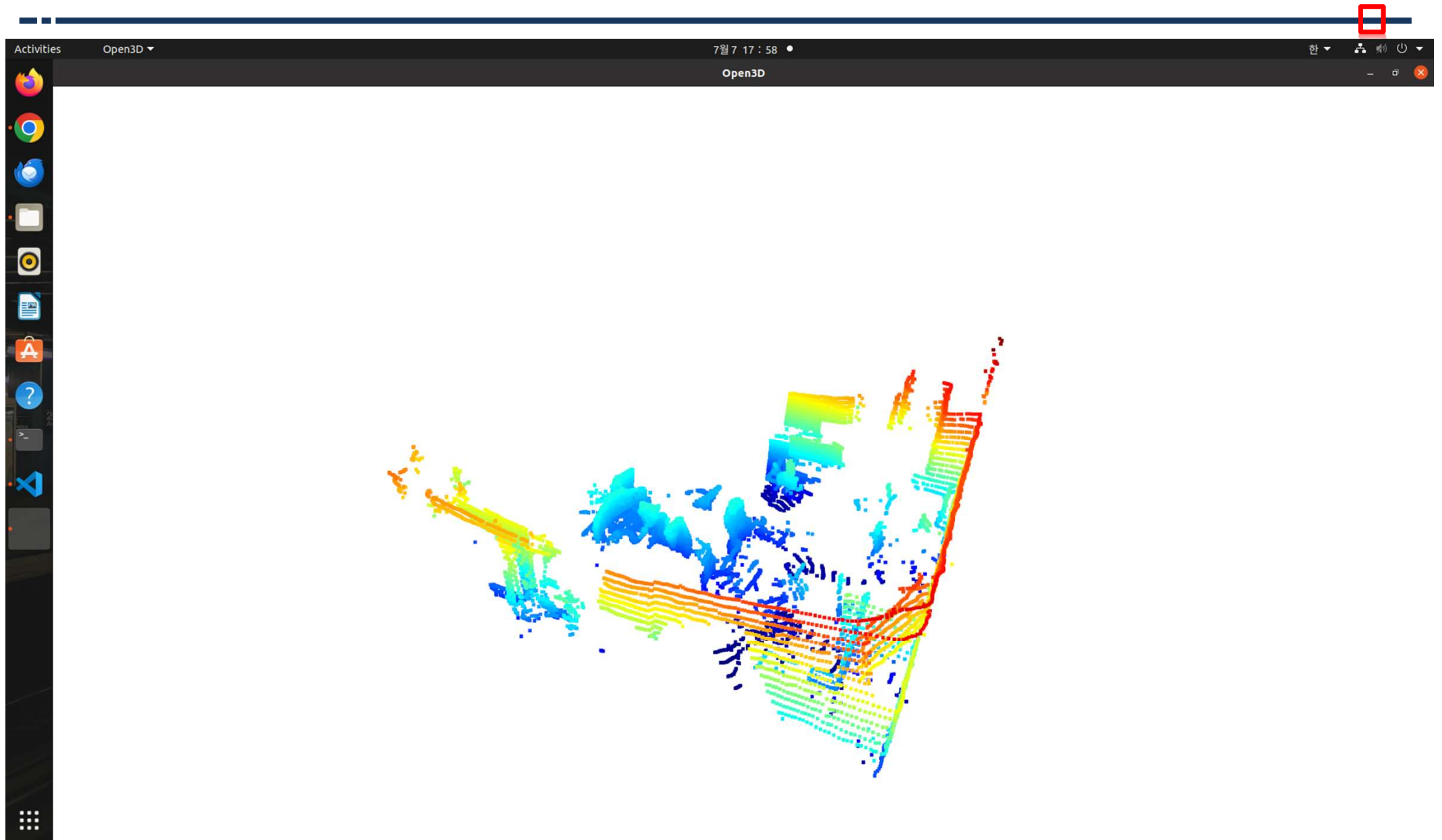
>> Get Sensor Data

3. Open3D

>> Process Point Cloud

Code Example 1

Ouster SDK & Open3D: Example



Visualization Example 1

Ouster SDK & Open3D: Example



```
an_live.py > ...
# Module Import
import open3d as o3d
import numpy as np
import time
from ouster.sdk import open_source
from ouster.sdk.core import XYZLut

# Get Sensor Info & MetaData
HOST = "192.168.0.49"
source = open_source(HOST)
metadata = source.metadata[0]
xyz_lut = XYZLut(metadata, False)

# Visualizer Start
vis = o3d.visualization.Visualizer()
vis.create_window(window_name='Live Ouster PointCloud', width=800, height=600)

# Visualizer Initialization
pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(np.zeros((1, 3))) # 초기 포인트 1개
vis.add_geometry(pcd)

# 바닥 평면 생성
ground_plane = o3d.geometry.TriangleMesh.create_box(width=4.0, height=0.01, depth=4.0)
# Z축을 위로 향하게 XY -> XZ 평면으로 회전
R = ground_plane.get_rotation_matrix_from_axis_angle([np.pi / 2, 0, 0]) # X축 기준 90도 회
ground_plane.rotate(R, center=(0, 0, 0)) # 원점 기준 회전
# 위치 조정 (살짝 아래에 깔리도록)
ground_plane.translate(np.array([-1, 2, -1.0]))
# 색상 설정
ground_plane.paint_uniform_color([0.5, 0.5, 0.5])
# 시각화에 추가
vis.add_geometry(ground_plane)

# 점 크기 설정
render_opt = vis.get_render_option()
render_opt.point_size = 3.0 # 여기서 조정

# first_frame
first_frame = True

# max_distance 설정
max_distance = 3 # 최대 거리 제한 (단위: 미터)

# 반복문을 돌려 Scan 시작
try:
    for scans in source:
        scan_live.py > ...
        # 반복문을 돌려 Scan 시작
        try:
            for scans in source:
                scan = scans[0]
                xyz = xyz_lut(scan) # (H, W, 3) 배열

                points = xyz.reshape(-1, 3)

                if points.shape[0] == 0:
                    continue # 포인트가 없으면 다음 프레임으로 (필수는 아님)

                # 거리 필터링: 원점 기준 max_distance 이상 점 제거
                distances = np.linalg.norm(points, axis=1)
                points = points[distances <= max_distance]

                if points.shape[0] == 0:
                    continue # 필터링 후 포인트 없으면 다음 프레임

                # 다운샘플링 추가
                pcd_temp = o3d.geometry.PointCloud()
                pcd_temp.points = o3d.utility.Vector3dVector(points)
                pcd_temp = pcd_temp.voxel_down_sample(voxel_size=0.01) # 크기 조절 여기서!

                # 노이즈 제거 추가 (Statistical Outlier Removal)
                pcd_temp, ind = pcd_temp.remove_statistical_outlier(nb_neighbors=25, std_ratio=2.0)

                # 평면 제거 (벽 탐지)
                # threshold 줄이거나 iteration 늘려서 더 넓은 거리 벽 제거
                plane_model, inliers = pcd_temp.segment_plane(distance_threshold=0.05,
                                                                ransac_n=3,
                                                                num_iterations=2000)

                [a, b, c, d] = plane_model
                normal = np.array([a, b, c])

                # 평면의 법선이 거의 수평이라면 -> 벽일 가능성 있음
                # 기준: Z축(상방향)과의 내적이 작으면 -> 수직 방향
                dot_with_z = np.abs(normal @ np.array([0, 0, 1]))

                # 수직 방향 (즉, 벽이라 판단되면 제거)
                if dot_with_z < 0.3: # Z축과 거의 수직
                    # 벽 인라이어 제외된 점만 사용
                    pcd_temp = pcd_temp.select_by_index(inliers, invert=True)

                pcd.points = pcd_temp.points

            vis.update_geometry(pcd)
            vis.poll_events()
            vis.update_renderer()

            # Error 방지를 위한 first_frame
            if first_frame:
                vis.reset_view_point(reset_bounding_box=True)
                first_frame = False

            # FPS 설정 (60프레임)
            time.sleep(1/60)

        finally:
            vis.destroy_window()
```

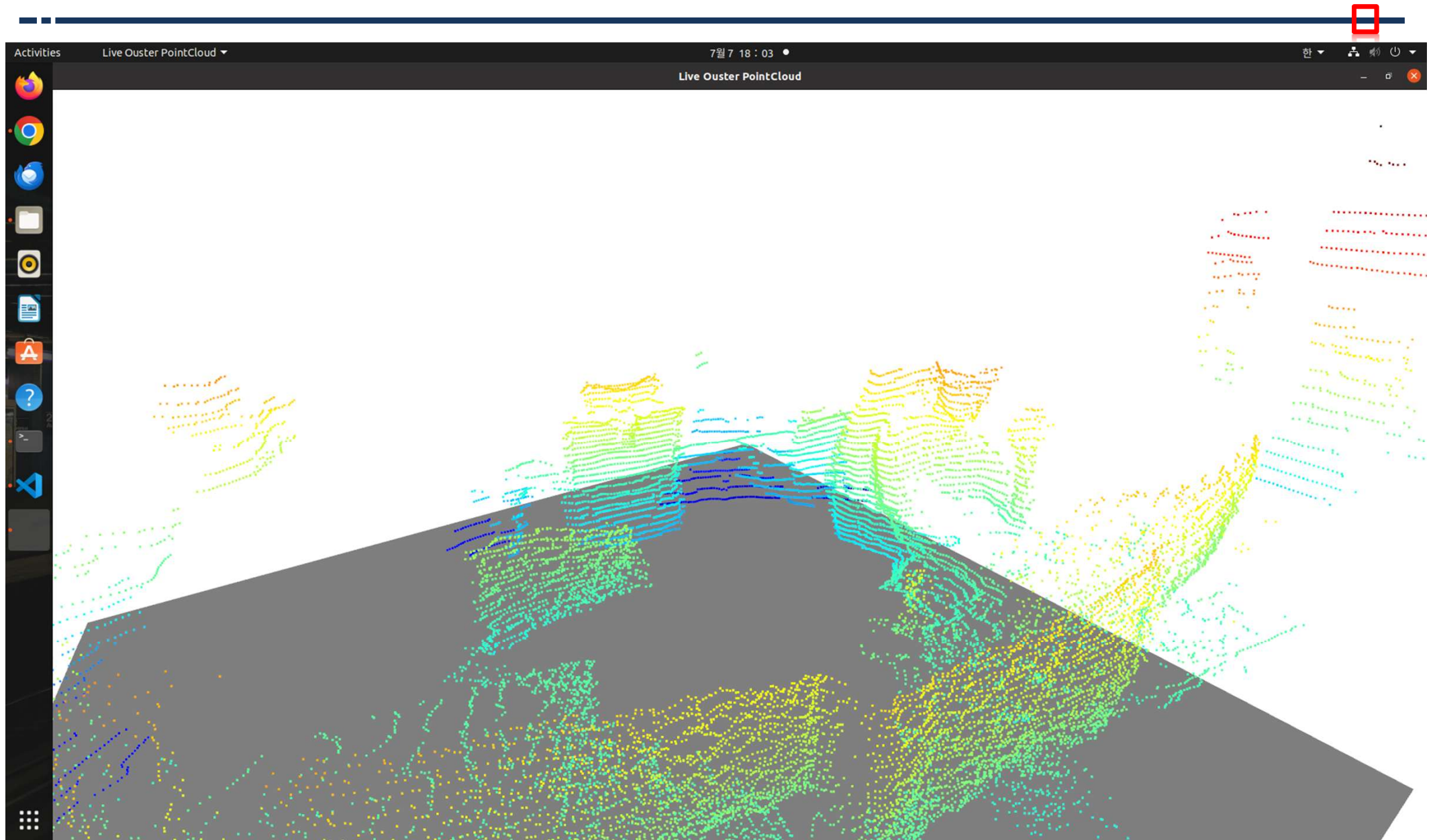
1. Get sensor data by using **ouster.sdk**

2. Process by using **open3D**

(Voxel downsampling, Noise Filtering, distance Limitation, etc)

Code Example 2

Ouster SDK & Open3D: Example



Visualization Example 2

Reference



- Ouster SDK
 - https://static.ouster.dev/sensor-docs/image_route1/image_route2/connecting/connecting-to-sensors.html
 - 센서값을 받기 위한 공식 SDK
- Ouster Studio
 - <https://ouster.com/products/software/ouster-studio>
 - 빠른 시각화 소프트웨어
- PCL: Point Cloud Library
 - <https://pointclouds.org/>
 - 포인트 클라우드 제어를 위한 C++ 라이브러리
- PointPillars
 - <https://github.com/zhulf0804/PointPillars>
 - 객체 인식을 위한 라이브러리
- Open3D
 - <https://www.open3d.org/docs/release/>
 - 포인트 클라우드 제어를 위한 Python 라이브러리

Thank you!

Questions?